

Q.No 9 :

Problem Description:

Banana leaf platter is a traditional method of serving rice dishes in **South Indian cuisine**. Due to the migration of South Indians, banana leaf rice can also be found in areas with significant ethnic South Indian diaspora such as **Malaysia** and **Singapore**.

Irfan is a banana leaf sales person.
he has N stacks of banana leaves.

Each stack contains K leaves.

Each leaf has a positive beauty value, describing how attractive it looks.

Irfan would like to take exactly P leaves to use for lunch today. If he would like to take a leaf in a stack, he must also take all of the leaves above it in that stack as well.

Help Irfan pick the P leaves that would maximize the total sum of attractive values.

Constraints:

$1 \leq T \leq 100$.

$1 \leq K \leq 30$.

$1 \leq P \leq N * K$.

$1 \leq N \leq 50$.

Input Format:

The first line of the input gives the number of test cases, T. T test cases follow. Each test case begins with a line containing the three integers N, K and

P. Then, N lines follow. The ith line contains K integers, describing the attractive values of each stack of leaves from top to bottom.

Output Format:

Print the output in a separate line contains the maximum total sum of attractive values that Irfan could pick.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
2
2 4 5
10 10 100 30
80 50 10 50
3 2 3
80 80
20 20
15 50
```

EXPECTED OUTPUT

```
250
180
```

Test Case 2

INPUT (STDIN)

```
2
2 4 5
20 20 70 30
60 30 10 50
3 2 3
60 70
15 40
20 10
```

EXPECTED OUTPUT

```
200
150
```

Code:

```
#include <stdio.h>

#define MAXN 50
#define MAXK 30
```

```
#define MAXP 1500

int max(int a, int b)
{
    if (a > b)
        return a;
    else
        return b;
}

int main()
{
    int T;
    scanf("%d", &T);

    int answer[100];

    for (int t = 0; t < T; t++)
    {
        int N, K, P;

        scanf("%d %d %d", &N, &K, &P);

        int dp[MAXP + 1];
        for (int i = 0; i <= P; i++)
        {
            dp[i] = -1000000000;
        }

        dp[0] = 0;

        for (int i = 0; i < N; i++)
        {
            int a[MAXK];
            int prefix[MAXK + 1];

            prefix[0] = 0;
            for (int j = 0; j < K; j++)
            {
                scanf("%d", &a[j]);
                prefix[j + 1] = prefix[j] + a[j];
            }

            int newdp[MAXP + 1];

            for (int j = 0; j <= P; j++)
            {
```

```

        newdp[j] = -1000000000;
    }
    for (int used = 0; used <= P; used++)
    {
        if (dp[used] == -1000000000)
            continue;

        for (int x = 0; x <= K; x++)
        {
            if (used + x <= P)
            {
                newdp[used + x] =
                    max(newdp[used + x],
                        dp[used] + prefix[x]);
            }
        }
    }
    for (int j = 0; j <= P; j++)
    {
        dp[j] = newdp[j];
    }
}

    answer[t] = dp[P];
}
printf("Output:\n");
for (int t = 0; t < T; t++)
{
    printf("%d\n", answer[t]);
}

return 0;
}

```

Output:

Test_case 1:

```
adityajaiswal@Adityas-MacBook-Air:~$ cat 1.txt
2
2 4 5
10 10 100 30
80 50 10 50
3 2 3
80 80
20 20
15 50
Output:
250
180
```

Test_case 2:

```
2
2 4 5
20 20 70 30
60 30 10 50
3 2 3
60 70
15 40
20 10
Output:
200
150
```